

Sri Lanka Institute of Information Technology



SE3090 – Software Engineering Frameworks
Year 3, Semester 1 - 2026

Practical Answer Submission

Practical Sheet No: 04

Student ID	IT24102798
Student Name	Sooriyabandara U.R.G.W.K.
Campus/ Center name	Malabe
Specialization	Artificial Intelligence
Batch No.	01.01

Task 01 – Design the Database Schema (3NF)

Table Designs

- **Users Table**

Column	Type	Key / Constraint
users.id	SERIAL / Integer	Primary Key
users.name	TEXT	NOT NULL
users.email	TEXT	NOT NULL, UNIQUE
users.password_hash	TEXT	NOT NULL
users.role	TEXT	NOT NULL, DEFAULT 'Customer'
users.created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

- **MenuItems Table**

Column	Type	Key / Constraint
menu_items.id	SERIAL / Integer	NOT NULL
menu_items.name	TEXT	NOT NULL
menu_items.price	NUMERIC(10,2)	NOT NULL
menu_items.category	TEXT	NOT NULL
menu_items.available	BOOLEAN	NOT NULL, DEFAULT TRUE

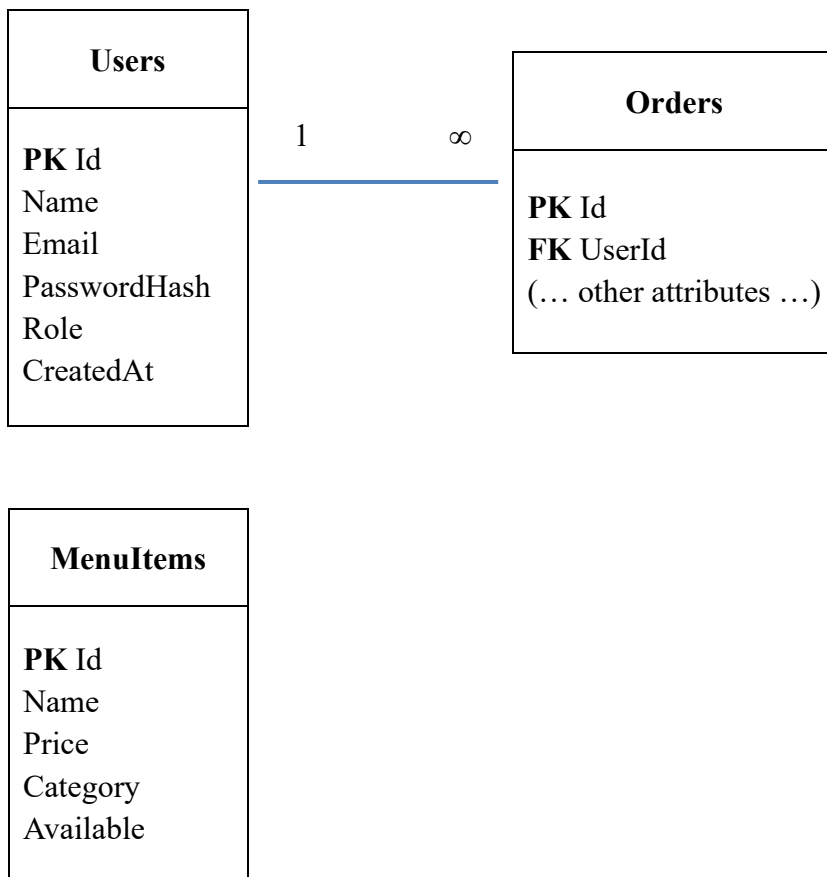
The FK-side answer for the User – Orders relationship

- One “User” can place many Orders, so the relationship is **one-to-many**.
- The foreign key is stored on the many side, which is the “Orders” table. Therefore, “Orders.UserId” will reference “Users.Id”.

The one-line 3NF justification per table

- **Users Table** : The Users table is in 3NF because all attributes are atomic and every non-key attribute depends directly on the primary key Id, with no partial or transitive dependencies.
- **MenuItems Table** : The MenuItems table is in 3NF because all attributes are atomic and every non-key attribute depends directly on the primary key Id, with no partial or transitive dependencies.

The ER sketch



Task 02 – Connect to PostgreSQL with EF Core

Screenshot of Successful Migration

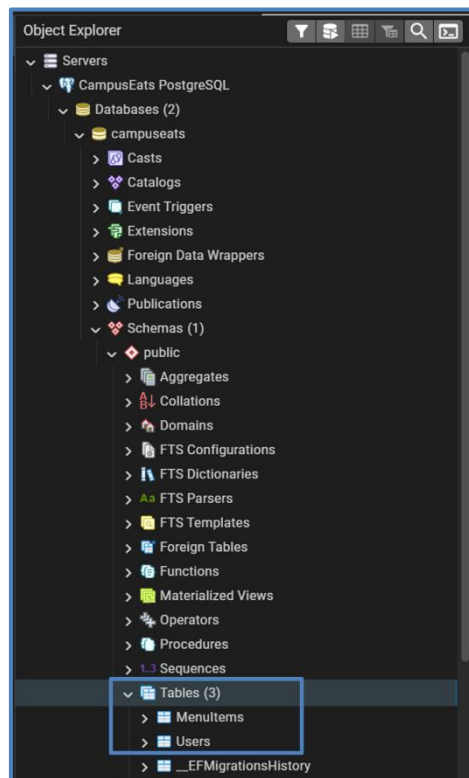
```
Terminal Command Prompt + - AI Agents : -
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

D:\SLIIT\V3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api>dotnet ef migration
s add InitialCreate
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'

D:\SLIIT\V3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api>dotnet ef database
update
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (28ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
        SELECT 1 FROM pg_catalog.pg_class c
        JOIN pg_catalog.pg_namespace n ON n.oid=c.relnamespace
        WHERE n.nspname='public' AND
              c.relname='__EFMigrationsHistory'
      )
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "__EFMigrationsHistory"
      ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
        SELECT 1 FROM pg_catalog.pg_class c
        JOIN pg_catalog.pg_namespace n ON n.oid=c.relnamespace
        WHERE n.nspname='public' AND
              c.relname='__EFMigrationsHistory'
      )
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
        SELECT 1 FROM pg_catalog.pg_class c
        JOIN pg_catalog.pg_namespace n ON n.oid=c.relnamespace
        WHERE n.nspname='public' AND
              c.relname='__EFMigrationsHistory'
      )
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "__EFMigrationsHistory"
      ORDER BY "MigrationId";
info: Microsoft.EntityFrameworkCore.Migrations[20402]
      Applying migration '20260810083444_InitialCreate'.
Applying migration '20260810083444_InitialCreate'.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (13ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE "MenuItems" (
        "Id" integer GENERATED BY DEFAULT AS IDENTITY,
        "Name" text NOT NULL,
        "Price" numeric NOT NULL,
        "Category" text NOT NULL,
        "Available" boolean NOT NULL,
        CONSTRAINT "PK_MenuItems" PRIMARY KEY ("Id")
      );
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE "Users" (
        "Id" integer GENERATED BY DEFAULT AS IDENTITY,
        "Name" text NOT NULL,
        "Email" text NOT NULL,
        "PasswordHash" text NOT NULL,
        "Role" text NOT NULL,
        "CreatedAt" timestamp with time zone NOT NULL,
        CONSTRAINT "PK_Users" PRIMARY KEY ("Id")
      );
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE UNIQUE INDEX "IX_Users_Email" ON "Users" ("Email");
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      INSERT INTO "__EFMigrationsHistory" ("MigrationId", "ProductVersion")
      VALUES ('20260810083444_InitialCreate', '8.0.29');
Done.

D:\SLIIT\V3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api>
```

Screenshot of Successful Table Creation of Users and MenuItems from pgAdmin



Screenshot of changes made to “MenuService” in “Program.cs”

```
C# Program.cs X
8 // Add services to the container.
9
10 builder.Services.AddControllers();
11 // Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
12 builder.Services.AddEndpointsApiExplorer();
13 builder.Services.AddSwaggerGen();
14 builder.Services.AddScoped<IMenuService, MenuService>();
15 builder.Services.AddDbContext<AppDbContext>(opt =>
16     opt.UseNpgsql(builder.Configuration // ConfigurationManager
17         .GetConnectionString( name: "Default")));
18
19 var app :WebApplication = builder.Build();
20
21 using (var scope = app.Services.CreateScope())
22 {
23     var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
24     if (!db.MenuItems.Any())
25     {
26         db.MenuItems.AddRange(
27             new MenuItem { Name = "Kottu Roti", Price = 750m, Category = "Mains" },
28             new MenuItem { Name = "Fried Rice", Price = 850m, Category = "Mains" },
29             new MenuItem { Name = "Watalappan", Price = 350m, Category = "Dessert" });
30         db.SaveChanges();
31     }
32 }
33
34 // Configure the HTTP request pipeline.
35 if (app.Environment.IsDevelopment())
36 {
37     app.UseSwagger();
38     app.UseSwaggerUI();
39 }
```

Screenshot of “GET /api/menu” which returns the three seeded dishes from PostgreSQL

The screenshot displays the Swagger UI for the CampusEats.Api v1. The selected definition is 'CampusEats.Api v1'. The endpoint being viewed is 'GET /api/Menu'.

Parameters: No parameters are defined for this endpoint.

Execute: A button to execute the request.

Responses:

200: The response body is a JSON array of three menu items:

```
{
  "id": 1,
  "name": "Kottu Rotl",
  "price": 100,
  "category": "Mains",
  "available": true
},
{
  "id": 2,
  "name": "Fried Rice",
  "price": 80,
  "category": "Mains",
  "available": true
},
{
  "id": 3,
  "name": "Matalappan",
  "price": 50,
  "category": "Dessert",
  "available": true
}
```

Response headers:

```
content-type: application/json; charset=utf-8
date: Mon, 18 Aug 2020 09:21:50 GMT
server: Kestrel
transfer-encoding: chunked
```

Media type: text/plain

Example Value:

```
{
  "id": 0,
  "name": "string",
  "price": 0,
  "category": "string",
  "available": true
}
```

POST /api/Menu

GET /api/Menu/{id}

PUT /api/Menu/{id}

DELETE /api/Menu/{id}

Schemas:

- CreateMenuitemDto
- MenuitemDto

Screenshot of the newly created Dish

The screenshot displays the Swagger UI for the **CampusEats.Api v1** API. The interface is in a dark theme. The top navigation bar shows the Swagger logo and the selected definition **CampusEats.Api v1**. The main content area is titled **Menu** and shows the **POST /api/Menu** endpoint. The endpoint is currently selected, and the **Parameters** tab is active, showing no parameters. The **Request body** is set to **application/json** and contains a JSON object:

```
{  "name": "Iced Milo",  "price": 400,  "category": "Drinks"}
```

. The **Execute** button is visible. Below the request body, the **Responses** section shows a **201** response with the following body:

```
{  "id": 4,  "name": "Iced Milo",  "price": 400,  "category": "Drinks",  "available": true}
```

. The **Response headers** are also displayed:

```
content-type: application/json; charset=utf-8
date: Mon, 10 Aug 2020 09:06:10 GMT
location: http://localhost:5000/api/Menu/4
server: Kestrel
transfer-encoding: chunked
```

. The **Responses** table shows a **200** response with the description **OK**. The **Media type** is set to **text/plain**. The **Example Value** is shown as a JSON object:

```
{  "id": 0,  "name": "string",  "price": 0,  "category": "string",  "available": true}
```

. The **Schemas** section at the bottom shows the **CreateMenuItemDto** and **MenuItemDto** schemas.

Swagger UI

localhost:5000/swagger/index.html

Select a definition CampusEats.Api v1

CampusEats.Api 1.0 OAS 3.0

http://localhost:5000/swagger/v1/swagger.json

Menu

GET /api/Menu

POST /api/Menu

Parameters

No parameters

Request body

application/json

```
{  "name": "Iced Milo",  "price": 400,  "category": "Drinks"}
```

Execute Clear

Responses

Curl

```
curl -X POST \  "http://localhost:5000/api/Menu" \  -H "accept: text/plain" \  -H "Content-Type: application/json" \  -d '{  "name": "Iced Milo",  "price": 400,  "category": "Drinks"  }'
```

Request URL

http://localhost:5000/api/Menu

Server response

Code Details

201

Response body

```
{  "id": 4,  "name": "Iced Milo",  "price": 400,  "category": "Drinks",  "available": true}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 10 Aug 2020 09:06:10 GMT
location: http://localhost:5000/api/Menu/4
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Controls Accept header

Example Value Schema

```
{  "id": 0,  "name": "string",  "price": 0,  "category": "string",  "available": true}
```

GET /api/Menu/{id}

PUT /api/Menu/{id}

DELETE /api/Menu/{id}

Schemas

CreateMenuItemDto >

MenuItemDto >

Screenshot of returning all Dishes before the Application Restart
(Time: Mon, 10 Aug 2026 09:07:12 GMT = 8/10/2026 2:37:12 PM)

The screenshot displays the Swagger UI for the CampusEats.Api v1. The selected definition is 'CampusEats.Api v1'. The endpoint being viewed is 'GET /api/Menu'. The response is a 200 status code with a JSON body containing a list of menu items. A blue arrow points to the 'Response headers' section, which shows the following headers:

```
content-type: application/json; charset=utf-8
date: Mon, 10 Aug 2026 09:07:12 GMT
server: Kestrel
transfer-encoding: chunked
```

The response body is a JSON array of menu items:

```
[{"id": 1, "name": "Hawaiian Roll", "price": 750, "category": "Main", "available": true}, {"id": 2, "name": "Fried Rice", "price": 850, "category": "Main", "available": true}, {"id": 3, "name": "Mentalappan", "price": 950, "category": "Dessert", "available": true}, {"id": 4, "name": "Iced Mllo", "price": 400, "category": "Drinks", "available": true}]
```

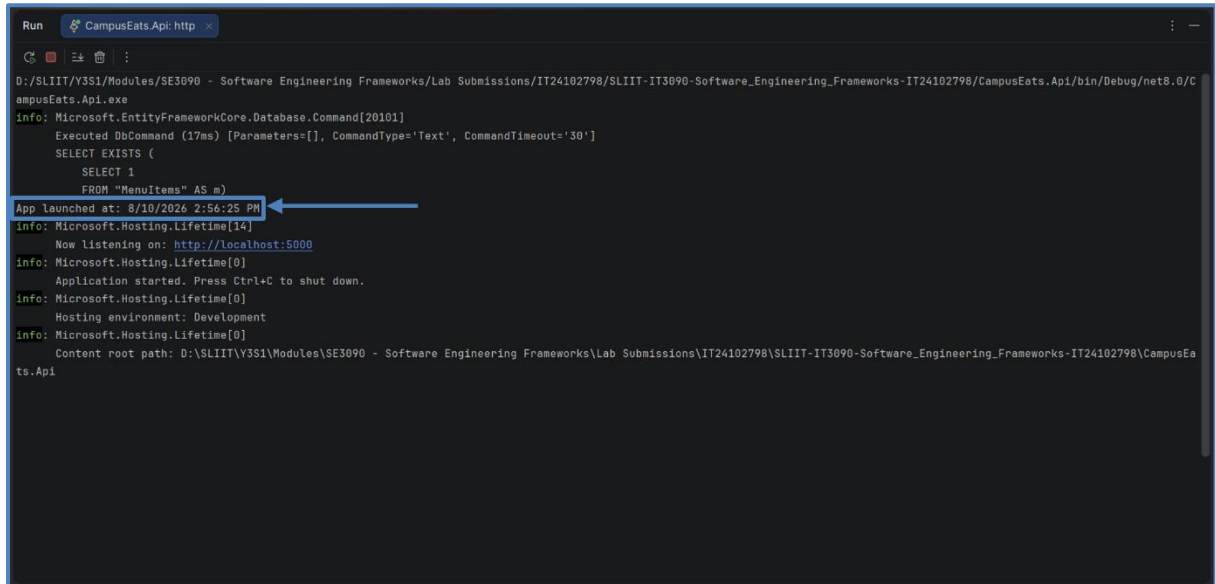
The 'Responses' section shows a 200 status code with a description of 'OK'. The 'Media type' is set to 'text/plain'. The 'Example Value' is a JSON object with the following structure:

```
{ "id": 1, "name": "string", "price": 0, "category": "string", "available": true }
```

The 'Schemas' section shows two schemas: 'CreateMenuitemDto' and 'MenuitemDto'.

Screenshot of the Application Restart

(Time: Mon, 10 Aug 2026 09:26:25 GMT = 8/10/2026 2:56:25 PM)



```
Run CampusEats.Api: http x
D:\SLIIT\Y3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api\bin\Debug\net8.0\CampusEats.Api.exe
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (17ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
        SELECT 1
        FROM "MenuItems" AS m)
App launched at: 8/10/2026 2:56:25 PM
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\SLIIT\Y3S1\Modules\SE3090 - Software Engineering Frameworks\Lab Submissions\IT24102798\SLIIT-IT3090-Software_Engineering_Frameworks-IT24102798\CampusEats.Api
```

Screenshot of returning all Dishes after the Application Restart
(Time: Mon, 10 Aug 2026 09:28:29 GMT = 8/10/2026 2:58:29 PM)

The screenshot displays the Swagger UI for the CampusEats.Api v1. The selected definition is 'CampusEats.Api v1'. The endpoint being viewed is 'GET /api/Menu'. The response status is 200. The response body is a JSON array of menu items, each with an id, name, price, category, and available status. The response headers are also visible, showing content-type, date, server, and transfer-encoding. A blue arrow points to the 'Response headers' section.

Menu

GET /api/Menu

Parameters

No parameters

Execute Clear

Responses

200

Response body

```
{
  "id": 1,
  "name": "Katte Butli",
  "price": 750,
  "category": "Main",
  "available": true
},
{
  "id": 2,
  "name": "Fried Rice",
  "price": 600,
  "category": "Main",
  "available": true
},
{
  "id": 3,
  "name": "Matalappan",
  "price": 700,
  "category": "Dessert",
  "available": true
},
{
  "id": 4,
  "name": "Iced Albu",
  "price": 400,
  "category": "Drinks",
  "available": true
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 10 Aug 2026 09:28:29 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type: text/plain

Content-accept Header:

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "price": 0,
  "category": "string",
  "available": true
}
```

Schemas

- CreateMenuItemDto >
- MenuItemDto >

Task 03 – User Registration with Password Hashing

Screenshot of the Working Registration Endpoint that creates a Customer

The screenshot displays the Swagger UI for the CampusEats.Api. The selected definition is CampusEats.Api v1. The endpoint being viewed is POST /api/Auth/register. The request body is a JSON object with the following fields: name (Sooriyabandara U.S.S.S.E.), email (124180798my.sllit.lk), and password (testPassword1234). The response is a 201 status code with a JSON body containing id, email, and role (customer). The response headers include content-type (application/json; charset=utf-8), date (Mon, 19 Aug 2024 11:48:18 GMT), server (Kestrel), and transfer-encoding (chunked).

Auth

POST /api/Auth/register

Parameters

No parameters

Request body

application/json

```
{
  "name": "Sooriyabandara U.S.S.S.E.",
  "email": "124180798my.sllit.lk",
  "password": "testPassword1234"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5000/api/Auth/register' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "name": "Sooriyabandara U.S.S.S.E.",
    "email": "124180798my.sllit.lk",
    "password": "testPassword1234"
  }'
```

Request URL

http://localhost:5000/api/Auth/register

Server response

Code Details

201
undocumented

Response body

```
{
  "id": 1,
  "email": "124180798my.sllit.lk",
  "role": "customer"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 19 Aug 2024 11:48:18 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Menu

GET /api/Menu

POST /api/Menu

GET /api/Menu/{id}

PUT /api/Menu/{id}

DELETE /api/Menu/{id}

Schemas

CreateMenuItemDto >

MenuItemDto >

RegisterDto >

Screenshot of the Users Table showing the Stored Hash (Never the Plaintext Password)

```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\WSI\AppData\Local\Programs\pgAdmin 4\runtime>"C:\Users\WSI\AppData\Local\Programs\pgAdmin 4\runtime\psql.exe" "host=localhost port=5432 dbname=campuseats user=postgres
sslmode=prefer connect_timeout=10 keepalives=1 keepalives_idle=30 keepalives_interval=10 keepalives_count=3" 2>>&1
psql (18.4)
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
Type "help" for help.

campuseats=# SELECT * FROM "Users";

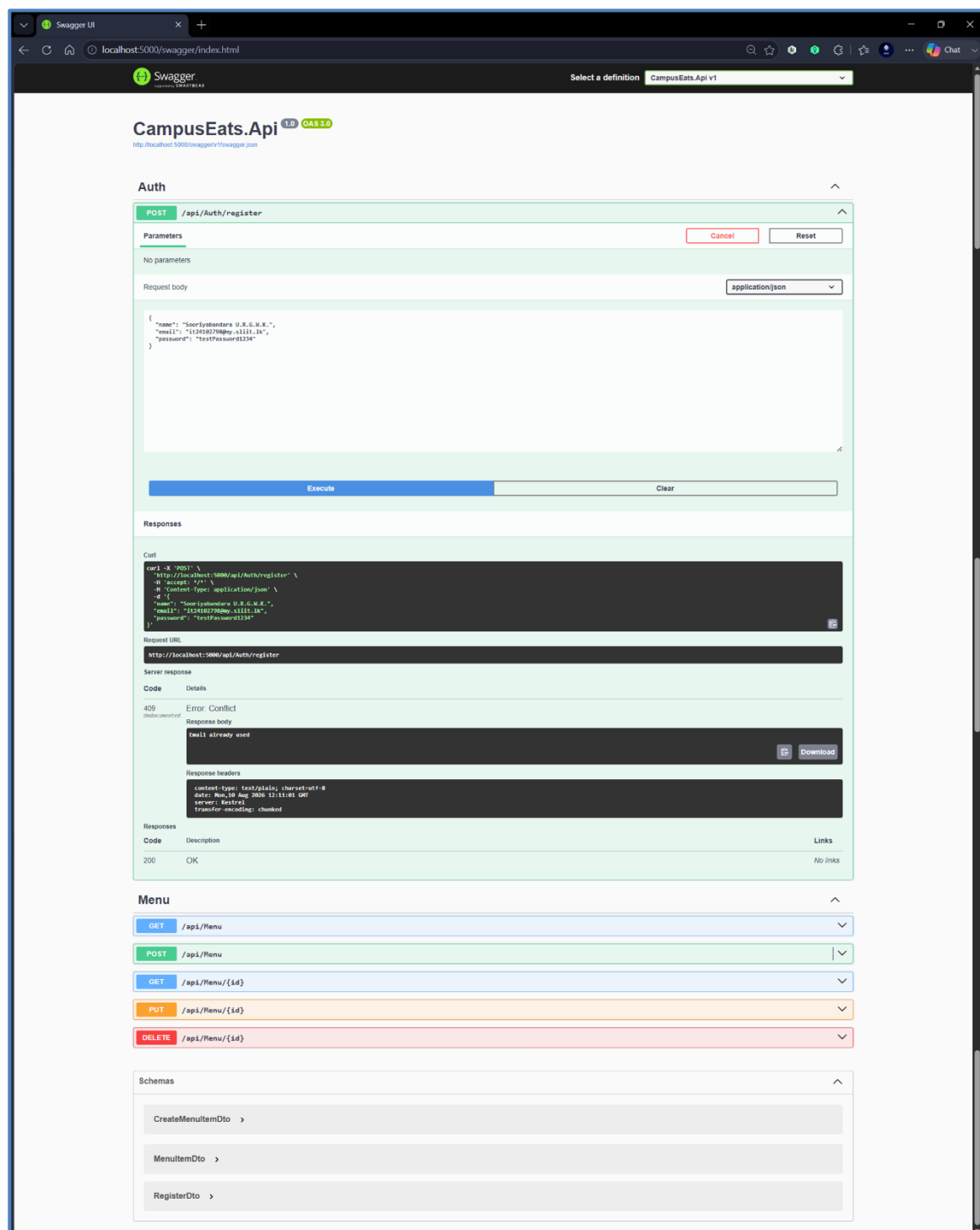
```

Id	Name	Email	PasswordHash	Role	CreatedAt
1	Sooriyabandara U.R.G.W.K.	it24102790@my.sliit.lk	\$2a\$11\$3WYRN14E.L1V0stYQ1exRuK1a9ggAqX0.MhtHg4.v32.VA3gFRaxW	Customer	2026-08-10 11:48:15.894732+00

```
(1 row)

campuseats=#
```

Screenshot of the 409 response when the same email is reused



Task 04 – Login & JWT Token Generation

Screenshot of Login API Endpoint returning a JWT for Valid Credentials

The screenshot displays the Swagger UI interface for the CampusEats.Api v1. The 'Auth' section is expanded, showing the 'POST /api/Auth/login' endpoint. The request body is set to 'application/json' and contains the following JSON:

```
{  "email": "t24t80798my.s111t-ik",  "password": "testPassword1234"}
```

The 'Execute' button is highlighted in blue. Below the request, the 'Responses' section shows a successful response with a status code of 200. The response body is a JSON object containing a 'token' field with a long JWT string. The response headers are also visible, showing 'content-type: application/json; charset=utf-8', 'date: Mon, 18 Aug 2020 14:13:20 GMT', 'server: Kestrel', and 'transfer-encoding: chunked'.

The 'Menu' section is also visible, showing endpoints for GET, POST, PUT, and DELETE requests to /api/Menu and /api/Menu/{id}.

The 'Schemas' section is expanded, showing the following schemas:

- CreateMenuItemDto
- LoginDto
- MenuItemDto
- RegisterDto

Screenshot of Login API Endpoint returning *401 Unauthorized* for Invalid Credentials

The screenshot displays the Swagger UI interface for the CampusEats.Api v1. The 'Auth' section is expanded, showing the POST endpoint /api/Auth/login. The request body is set to application/json with the following payload:

```
{  "email": "t24182798my.xlitt.lk",  "password": "wrongPassword"}
```

The 'Execute' button has been clicked, resulting in a 401 Unauthorized response. The response body is 'Invalid credentials'. The response headers are:

```
content-type: text/plain; charset=utf-8
date: Mon, 10 Aug 2020 14:44:54 GMT
server: Node.js
transfer-encoding: chunked
```

The 'Responses' section shows a table with the following data:

Code	Description	Links
200	OK	No links

The 'Menu' section is also visible, showing endpoints for /api/Menu, /api/Menu/{id}, and /api/Menu/{id} with methods GET, POST, PUT, and DELETE.

The 'Schemas' section is expanded, showing the following schemas:

- CreateMenuItemDto
- LoginDto
- MenuItemDto
- RegisterDto

Screenshot of the Token decoded at *jwt.io* showing the Role Claim

The screenshot displays the JWT Debugger web application. The interface is dark-themed and includes a navigation bar with links to 'Debugger', 'Introduction', 'Libraries', and 'Ask'. The main heading is 'JSON Web Token (JWT) Debugger'. Below this, a subheading states: 'Decode, verify, and generate JSON Web Tokens, which are an open, industry standard RFC 7519 method for representing claims securely between two parties.'

The 'JWT Decoder' tab is active. It features a text input field for pasting a JWT token, a 'Generate example' button, and a 'Select signing algorithm' dropdown. The 'Encoded Token' field contains a long base64-encoded string. Below it, a 'Valid JWT' status is shown. The 'Decoded Header' section displays the header claims: 'alg': 'HS256' and 'typ': 'JWT'. The 'Decoded Payload' section displays the payload claims, including 'http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress': 't24102798@my.slist.it', 'http://schemas.microsoft.com/ws/2008/06/identity/claims/role': 'Customer', 'exp': 1786379180, 'iss': 'CampusData', and 'aud': 'CampusDataUsers'. The 'JWT Signature Verification (Optional)' section is also visible, with a 'Secret' input field and a 'signature verification failed' message.

One-Line Note on what the Signature is for,

- The JWT signature verifies that the token was issued by the server and that its contents have not been modified or forged.

Task 05 – Protect Endpoints with RBAC & Test the Secure Flow

Screenshot of POST /api/Menu with 401 Unauthorized for accessing without a Token

The screenshot displays the Swagger UI for the CampusEats.Api (v1). The interface is in a dark theme. The 'Menu' section is expanded, showing the POST endpoint /api/Menu. The request body is set to application/json with the following payload:

```
{  "name": "Hot Chocolate",  "price": 450,  "category": "Drinks"}
```

The 'Execute' button is highlighted in blue. Below the request, the 'Responses' section shows the result of the request:

401 Error: Unauthorized

Response headers:

```
content-length: 0
date: Mon, 18 Aug 2020 15:14:53 GMT
server: Kestrel
www-authenticate: Bearer
```

The 'Responses' table shows the following details:

Code	Description	Links
200	OK	No links

The 'Schemas' section at the bottom lists the following DTOs:

- CreateMenuItemDto
- LoginDto
- MenuItemDto
- RegisterDto

Screenshot of POST /api/Menu with 403 Forbidden for accessing with Customer JWT Token

The screenshot displays the Swagger UI for the CampusEats.Api v1. The interface is in a dark theme. The top navigation bar shows the Swagger logo and the API definition selected. The main content area is divided into sections: Auth, Menu, and Schemas. The Menu section is expanded, showing the POST /api/Menu endpoint. The endpoint is currently selected, and the 'Execute' button is visible. The 'Request body' is set to 'application/json' and contains a JSON object: { "name": "Hot Chocolate", "price": 4.99, "category": "Drinks" }. The 'Responses' section shows a 403 Error Forbidden response. The response headers are: content-length: 0, date: Mon, 18 Aug 2025 15:17:52 GMT, server: Kestrel. The response body is empty. The 'Schemas' section at the bottom lists several data models: CreateMenuItemDto, LoginDto, MenuItemDto, and RegisterDto.

Swagger UI interface showing the API definition for CampusEats.Api v1. The selected endpoint is POST /api/Menu. The request body is application/json, and the response is 403 Error Forbidden.

Auth

- POST /api/Auth/register
- POST /api/Auth/login

Menu

- GET /api/Menu
- POST /api/Menu

Parameters

No parameters

Request body

application/json

```
{  "name": "Hot Chocolate",  "price": 4.99,  "category": "Drinks"}
```

Responses

Curl

```
curl -X POST -H "Host: localhost:5000" -H "Content-Type: application/json" -d '{"name": "Hot Chocolate", "price": 4.99, "category": "Drinks"}' http://localhost:5000/api/Menu
```

Request URL

http://localhost:5000/api/Menu

Server response

Code **Details**

403 Error Forbidden

Response headers

```
content-length: 0date: Mon, 18 Aug 2025 15:17:52 GMTserver: Kestrel
```

Responses

Code	Description	Links
200	OK	No links

Schemas

- CreateMenuItemDto
- LoginDto
- MenuItemDto
- RegisterDto

Screenshot of POST /api/Menu with 201 Created for accessing with Admin JWT Token

The screenshot displays the Swagger UI for the CampusEats.Api v1. The interface is in a dark theme. The top navigation bar shows the Swagger logo and the API definition selected. The main content area is divided into sections: Auth, Menu, and Responses.

Auth Section:

- POST /api/Auth/register
- POST /api/Auth/login

Menu Section:

- GET /api/Menu
- POST /api/Menu (selected)

POST /api/Menu Details:

- Parameters:** No parameters.
- Request body:** application/json. The body contains a JSON object:

```
{  "name": "Hot Chocolate",  "price": 400,  "category": "Drinks"}
```
- Execute:** A blue button to execute the request.

Responses Section:

- 201 Created:** The response body is a JSON object:

```
{  "id": 1,  "name": "Hot Chocolate",  "price": 400,  "category": "Drinks",  "available": true}
```

 The response headers are:

```
content-type: application/json; charset=utf-8date: Mon, 19 Aug 2020 15:29:22 GMTlocation: http://localhost:5000/api/Menu/5server: Kestreltransfer-encoding: chunked
```
- 200 OK:** The response body is a JSON object:

```
{  "id": 1,  "name": "string",  "price": 400,  "category": "string",  "available": true}
```

Schemas Section:

- CreateMenuitemDto
- LoginDto
- MenuitemDto
- RegisterDto

Short Written Justifications

- **Why store a password hash instead of the password?**
 - A password hash is stored instead of the plaintext password because a one-way hash prevents the original password from being directly recovered from the database. BCrypt also uses salting, making password hashes harder to crack even if the database is compromised.
- **Why use a stateless JWT instead of a server-side session?**
 - A stateless JWT allows the server to authenticate requests using the information contained in the token without maintaining a session record for every logged-in user. This makes the API easier to scale because requests can be handled independently by different server instances as long as they can validate the token.
- **Why use role-based access control instead of checking individual users in every method?**
 - Role-based access control groups users according to their responsibilities, such as Customer and Admin, instead of hard-coding individual user checks into every endpoint. This makes authorization easier to maintain and allows the same access rules to be applied consistently to many users and endpoints.

Quick Test – Knowledge Check

Multiple Choice (Choose One)

Q1. (c) primary key

Q2. (b) the Orders (many) side

Q3. (c) 403 Forbidden

Q4. (b) as a salted one-way hash

Q5. (c) signature

Database Design

Q6.

- It breaks First Normal Form (1NF) because the course columns represent repeating groups instead of storing one atomic value per field. It is better to be redesigned using separate Students, Courses, and StudentCourses tables, where StudentCourses contain the foreign keys StudentId and CourseId to represent the many-to-many relationship.

Authentication

Q7.

- **Step 01:** The user submits their email and password to the login endpoint, and the server verifies the password against the stored BCrypt hash.
- **Step 02:** If the credentials are valid, the server creates and signs a JWT containing claims such as the user's ID, email, and role.
- **Step 03:** The server returns the JWT to the client, which stores it and sends it with future authenticated requests as a Bearer token.

Authorization Scenario

Q8.

- It is 403 Forbidden because the Customer is already authenticated with a valid JWT, but their Customer role is not permitted to delete menu items. The endpoint is protected by [Authorize(Roles = "Admin")], which allows only users with the Admin role to perform the operation.